

Introduction to Assembly Language

1.1. BASIC COMPUTER ARCHITECTURE

Address, Data, and Control Buses

A computer system comprises of a processor, memory, and I/O devices. I/O is used for interfacing with the external world, while memory is the processor's internal world. Processor is the core in this picture and is responsible for performing operations. The operation of a computer can be fairly described with processor and memory only. I/O will be discussed in a later part of the course. Now the whole working of the computer is performing an operation by the processor on data, which resides in memory.

The scenario that the processor executes operations and the memory contains data elements requires a mechanism for the processor to read that data from the memory. "That data" in the previous sentence much be rigorously explained to the memory which is a dumb device. Just like a postman, who must be told the precise address on the letter, to inform him where the destination is located. Another significant point is that if we only want to read the data and not write it, then there must be a mechanism to inform the memory that we are interested in reading data and not writing it. Key points in the above discussion are:

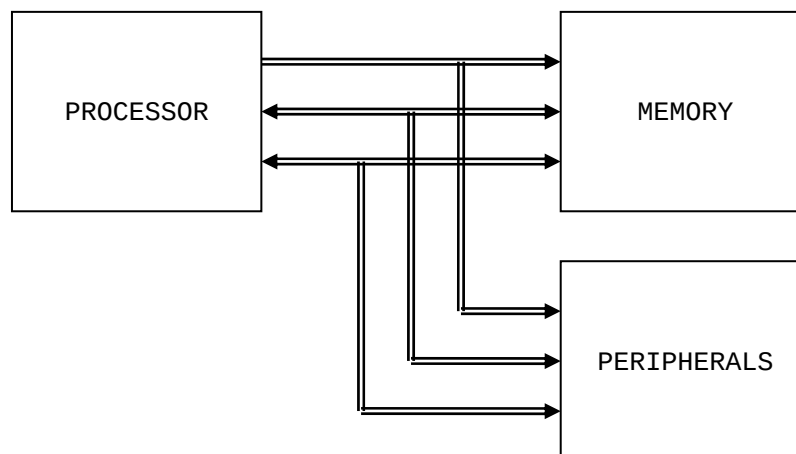
- There must be a mechanism to inform memory that we want to do the read operation
- There must be a mechanism to inform memory that we want to read precisely which element
- There must be a mechanism to transfer that data element from memory to processor

The group of bits that the processor uses to inform the memory about which element to read or write is collectively known as the *address bus*. Another important bus called the *data bus* is used to move the data from the memory to the processor in a read operation and from the processor to the memory in a write operation. The third group consists of miscellaneous independent lines used for control purposes. For example, one line of the bus is used to inform the memory about whether to do the read operation or the write operation. These lines are collectively known as the *control bus*.

These three buses are the eyes, nose, and ears of the processor. It uses them in a synchronized manner to perform a meaningful operation. Although the programmer specifies the meaningful operation, but to fulfill it the processor needs the collaboration of other units and peripherals. And that collaboration is made available using the three buses. This is the very basic description of a computer and it can be extended on the same lines to I/O but we are leaving it out just for simplicity for the moment.

The address bus is unidirectional and address always travels from processor to memory. This is because memory is a dumb device and cannot predict which element the processor at a particular instant of time needs. Data moves from both, processor to memory and memory to processor, so the data bus is bidirectional. Control bus is special and relatively complex, because different lines comprising it behave differently. Some take

information from the processor to a peripheral and some take information from the peripheral to the processor. There can be certain events outside the processor that are of its interest. To bring information about these events the data bus cannot be used as it is owned by the processor and will only be used when the processor grants permission to use it. Therefore certain processors provide control lines to bring such information to processor's notice in the control bus. Knowing these signals in detail is unnecessary but the general idea of the control bus must be conceived in full.



We take an example to explain the collaboration of the processor and memory using the address, control, and data buses. Consider that you want your uneducated servant to bring a book from the shelf. You order him to bring the fifth book from top of the shelf. All the data movement operations are hidden in this one sentence. Such a simple everyday phenomenon seen from this perspective explains the seemingly complex working of the three buses. We told the servant to “bring a book” and the one which is “fifth from top,” precise location even for the servant who is much more intelligent than our dumb memory. The dumb servant follows the steps one by one and the book is in your hand as a result. If however you just asked him for a book or you named the book, your uneducated servant will stand there gazing at you and the book will never come in your hand.

Even in this simplest of all examples, mathematics is there, “fifth from top.” Without a number the servant would not be able to locate the book. He is unable to understand your will. Then you tell him to put it with the seventh book on the right shelf. Precision is involved and only numbers are precise in this world. One will always be one and two will always be two. So we tell in the form of a number on the address bus which cell is needed out of say the 2000 cells in the whole memory.

A binary number is generated on the address bus, fifth, seventh, eighth, tenth; the cell which is needed. So the cell number is placed on the address bus. A memory cell is an n-bit location to store data, normally 8-bit also called a byte. The number of bits in a cell is called the *cell width*. The two dimensions, cell width and number of cells, define the memory completely just like the width and depth of a well defines it completely. 200 feet deep by 15 feet wide and the well is completely described. Similarly for memory we define two dimensions. The first dimension defines how many parallel bits are there in a single memory cell. The memory is called 8-bit or 16-bit for this reason and this is also the word size of the memory. This need not match the size of a processor word which has other parameters to define it. In general the memory cell cannot be wider than the width of the data bus. Best and simplest operation requires the same size of data bus and memory cell width.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

As we previously discussed that the control bus carries the intent of the processor that it wants to read or to write. Memory changes its behavior in response to this signal from the processor. It defines the direction of data flow. If processor wants to read but memory wants to write, there will be no communication or useful flow of information. Both must be synchronized, like a speaker speaks and the listener listens. If both speak simultaneously or both listen there will be no communication. This precise synchronization between the processor and the memory is the responsibility of the control bus.

Control bus is only the mechanism. The responsibility of sending the appropriate signals on the control bus to the memory is of the processor. Since the memory never wants to listen or to speak of itself. Then why is the control bus bidirectional. Again we take the same example of the servant and the book further to elaborate this situation. Consider that the servant went to fetch the book just to find that the drawing room door is locked. Now the servant can wait there indefinitely keeping us in surprise or come back and inform us about the situation so that we can act accordingly. The servant even though he was obedient was unable to fulfill our orders so in all his obedience, he came back to inform us about the problem. Synchronization is still important, as a result of our orders either we got the desired cell or we came to know that the memory is locked for the moment. Such information cannot be transferred via the address or the data bus. For such situations when peripherals want to talk to the processor when the processor wasn't expecting them to speak, special lines in the control bus are used. The information in such signals is usually to indicate the incapability of the peripheral to do something for the moment. For these reasons the control bus is a bidirectional bus and can carry information from processor to memory as well as from memory to processor.

1.2. REGISTERS

The basic purpose of a computer is to perform operations, and operations need operands. Operands are the data on which we want to perform a certain operation. Consider the addition operation; it involves adding two numbers to get their sum. We can have precisely one address on the address bus and consequently precisely one element on the data bus. At the very same instant the second operand cannot be brought inside the processor. As soon as the second is selected, the first operand is no longer there. For this reason there are temporary storage places inside the processor called *registers*. Now one operand can be read in a register and added into the other which is read directly from the memory. Both are made accessible at one instance of time, one from inside the processor and one from outside on the data bus. The result can be written to at a distinct location as the operation has completed and we can access a different memory cell. Sometimes we hold both operands in registers for the sake of efficiency as what we can do inside the processor is undoubtedly faster than if we have to go outside and bring the second operand.

Registers are like a scratch pad ram inside the processor and their operation is very much like normal memory cells. They have precise locations and remember what is placed inside them. They are used when we need more than one data element inside the processor at one time. The concept of registers will be further elaborated as we progress into writing our first program.

Memory is a limited resource but the number of memory cells is large. Registers are relatively very small in number, and are therefore a very scarce and precious resource. Registers are more than one in number, so we have to precisely identify or name them. Some manufacturers number their registers like r0, r1, r2, others name them like A, B, C, D etc. Naming is useful since the registers are few in number. This is called the nomenclature of the

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

particular architecture. Still other manufacturers name their registers according to their function like X stands for an index register. This also informs us that there are special functions of registers as well, some of which are closely associated to the particular architecture. For example index registers do not hold data instead they are used to hold the address of data. There are other functions as well and the whole spectrum of register functionalities is quite large. However most of the details will become clear as the registers of the Intel architecture are discussed in detail.

Accumulator

There is a central register in every processor called the accumulator. Traditionally all mathematical and logical operations are performed on the accumulator. The word size of a processor is defined by the width of its accumulator. A 32bit processor has an accumulator of 32 bits.

Pointer, Index, or Base Register

The name varies from manufacturer to manufacturer, but the basic distinguishing property is that it does not hold data but holds the address of data. The rationale can be understood by examining a “for” loop in a higher level language, zeroing elements in an array of ten elements located in consecutive memory cells. The location to be zeroed changes every iteration. That is the address where the operation is performed is changing. Index register is used in such a situation to hold the address of the current array location. Now the value in the index register cannot be treated as data, but it is the address of data. In general whenever we need access to a memory location whose address is not known until runtime we need an index register. Without this register we would have needed to explicitly code each iteration separately.

In newer architectures the distinction between accumulator and index registers has become vague. They have general registers which are more versatile and can do both functions. They do have some specialized behaviors but basic operations can be done on all general registers.

Flags Register or Program Status Word

This is a special register in every architecture called the flags register or the program status word. Like the accumulator it is an 8, 16, or 32 bits register but unlike the accumulator it is meaningless as a unit, rather the individual bits carry different meanings. The bits of the accumulator work in parallel as a unit and each bit mean the same thing. The bits of the flags register work independently and individually, and combined its value is meaningless.

An example of a bit commonly present in the flags register is the carry flag. The carry can be contained in a single bit as in binary arithmetic the carry can only be zero or one. If a 16bit number is added to a 16bit accumulator, and the result is of 17 bits the 17th bit is placed in the carry bit of the flags register. Without this 17th bit the answer is incorrect. More examples of flags will be discussed when dealing with the Intel specific register set.

Program Counter or Instruction Pointer

Everything must translate into a binary number for our dumb processor to understand it, be it an operand or an operation itself. Therefore the instructions themselves must be translated into numbers. For example to add numbers we understand the word “add.” We translate this word into a number to make the processor understand it. This number is the actual instruction for the computer. All the objects, inheritance and encapsulation constructs in higher level languages translate down to just a number in assembly language in the end. Addition, multiplication, shifting; all big

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

programs are made using these simple building blocks. A number is at the bottom line since this is the only thing a computer can understand.

A program is defined to be “an ordered set of instructions.” Order in this definition is a key part. Instructions run one after another, first, second, third and so on. Instructions have a positional relationship. The whole logic depends on this positioning. If the computer executes the fifth instructions after the first and not the second, all our logic is gone. The processor should ensure this ordering of instructions. A special register exists in every processor called the program counter or the instruction pointer that ensures this ordering. “The program counter holds the address of the next instruction to be executed.” A number is placed in the memory cell pointed to by this register and that number tells the processor which instruction to execute; for example 0xEA, 255, or 152. For the processor 152 might be the add instruction. Just this one number tells it that it has to add, where its operands are, and where to store the result. This number is called the *opcode*. The instruction pointer moves from one opcode to the next. This is how our program executes and progresses. One instruction is picked, its operands are read and the instruction is executed, then the next instruction is picked from the new address in instruction pointer and so on.

Remembering 152 for the add operation or 153 for the subtract operation is difficult. To make a simple way to remember difficult things we associate a symbol to every number. As when we write “add” everyone understands what we mean by it. Then we need a small program to convert this “add” of ours to 152 for the processor. Just a simple search and replace operation to translate all such symbols to their corresponding opcodes. We have mapped the numeric world of the processor to our symbolic world. “Add” conveys a meaning to us but the number 152 does not. We can say that add is closer to the programmer’s thinking. This is the basic motive of adding more and more translation layers up to higher level languages like C++ and Java and Visual Basic. These symbols are called *instruction mnemonics*. Therefore the mnemonic “add a to b” conveys more information to the reader. The dumb translator that will convert these mnemonics back to the original opcodes is a key program to be used throughout this course and is called the *assembler*.

1.3. INSTRUCTION GROUPS

Usual opcodes in every processor exist for moving data, arithmetic and logical manipulations etc. However their mnemonics vary depending on the will of the manufacturer. Some manufacturers name the mnemonics for data movement instructions as “move,” some call it “load” and “store” and still other names are present. But the basic set of instructions is similar in every processor. A grouping of these instructions makes learning a new processor quick and easy. Just the group an instruction belongs tells a lot about the instruction.

Data Movement Instructions

These instructions are used to move data from one place to another. These places can be registers, memory, or even inside peripheral devices. Some examples are:

```
mov ax, bx
lad 1234
```

Arithmetic and Logic Instructions

Arithmetic instructions like addition, subtraction, multiplication, division and Logical instructions like logical and, logical or, logical xor, or complement are part of this group. Some examples are:

```
and ax, 1234
add bx, 0534
add bx, [1200]
```

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

The bracketed form is a complex variation meaning to add the data placed at address 1200. Addressing data in memory is a detailed topic and is discussed in the next chapter.

Program Control Instructions

The instruction pointer points to the next instruction and instructions run one after the other with the help of this register. We can say that the instructions are tied with one another. In some situations we don't want to follow this implied path and want to order the processor to break its flow if some condition becomes true instead of the spatially placed next instruction. In certain other cases we want the processor to first execute a separate block of code and then come back to resume processing where it left.

These are instructions that control the program execution and flow by playing with the instruction pointer and altering its normal behavior to point to the next instruction. Some examples are:

```
cmp ax, 0
jne 1234
```

We are changing the program flow to the instruction at 1234 address if the condition that we checked becomes true.

Special Instructions

Another group called special instructions works like the special service commandos. They allow changing specific processor behaviors and are used to play with it. They are used rarely but are certainly used in any meaningful program. Some examples are:

```
cli
sti
```

Where cli clears the interrupt flag and sti sets it. Without delving deep into it, consider that the cli instruction instructs the processor to close its ears from the outside world and never listen to what is happening outside, possibly to do some very important task at hand, while sti restores normal behavior. Since these instructions change the processor behavior they are placed in the special instructions group.

1.4. INTEL IAPX88 ARCHITECTURE

Now we select a specific architecture to discuss these abstract ideas in concrete form. We will be using IBM PC based on Intel architecture because of its wide availability, because of free assemblers and debuggers available for it, and because of its wide use in a variety of domains. However the concepts discussed will be applicable on any other architecture as well; just the mnemonics of the particular language will be different.

Technically iAPX88 stands for "Intel Advanced Processor Extensions 88." It was a very successful processor also called 8088 and was used in the very first IBM PC machines. Our discussion will revolve around 8088 in the first half of the course while in the second half we will use iAPX386 which is very advanced and powerful processor. 8088 is a 16bit processor with its accumulator and all registers of 16 bits. 386 on the other hand, is a 32bit processor. However it is downward compatible with iAPX88 meaning that all code written for 8088 is valid on the 386. The architecture of a processor means the organization and functionalities of the registers it contains and the instructions that are valid on the processor. We will discuss the register architecture of 8088 in detail below while its instructions are discussed in the rest of the book at appropriate places.

1.5. HISTORY

Intel did release some 4bit processors in the beginning but the first meaningful processor was 8080, an 8bit processor. The processor became

popular due to its simplistic design and versatile architecture. Based on the experience gained from 8080, an advanced version was released as 8085. The processor became widely popular in the engineering community again due to its simple and logical nature.

Intel introduced the first 16bit processor named 8088 at a time when the concept of personal computer was evolving. With a maximum memory of 64K on the 8085, the 8088 allowed a whole mega byte. IBM embedded this processor in their personal computer. The first machines ran at 4.43 MHz; a blazing speed at that time. This was the right thing at the right moment. No one expected this to become the biggest success of computing history. IBM PC XT became so popular and successful due to its open architecture and easily available information.

The success was unexpected for the developers themselves. As when Intel introduced the processor it contained a timer tick count which was valid for five years only. They never anticipated the architecture to stay around for more than five years but the history took a turn and the architecture is there at every desk even after 25 years and the tick is to be specially handled every now and then.

1.6. REGISTER ARCHITECTURE

The iAPX88 architecture consists of 14 registers.

CS	SP	
DS	BP	
SS	SI	
ES	DI	
	AH	AL (AX)
IP	BH	BL (BX)
	CH	CL (CX)
FLAGS	DH	DL (DX)

General Registers (AX, BX, CX, and DX)

The registers AX, BX, CX, and DX behave as general purpose registers in Intel architecture and do some specific functions in addition to it. X in their names stand for extended meaning 16bit registers. For example AX means we are referring to the extended 16bit “A” register. Its upper and lower byte are separately accessible as AH (A high byte) and AL (A low byte). All general purpose registers can be accessed as one 16bit register or as two 8bit registers. The two registers AH and AL are part of the big whole AX. Any change in AH or AL is reflected in AX as well. AX is a composite or extended register formed by gluing together the two parts AH and AL.

The A of AX stands for Accumulator. Even though all general purpose registers can act as accumulator in most instructions there are some specific variations which can only work on AX which is why it is named the accumulator. The B of BX stands for Base because of its role in memory addressing as discussed in the next chapter. The C of CX stands for Counter as there are certain instructions that work with an automatic count in the CX register. The D of DX stands for Destination as it acts as the destination in I/O operations. The A, B, C, and D are in letter sequence as well as depict some special functionality of the register.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

Index Registers (SI and DI)

SI and DI stand for source index and destination index respectively. These are the index registers of the Intel architecture which hold address of data and used in memory access. Being an open and flexible architecture, Intel allows many mathematical and logical operations on these registers as well like the general registers. The source and destination are named because of their implied functionality as the source or the destination in a special class of instructions called the string instructions. However their use is not at all restricted to string instructions. SI and DI are 16bit and cannot be used as 8bit register pairs like AX, BX, CX, and DX.

Instruction Pointer (IP)

This is the special register containing the address of the next instruction to be executed. No mathematics or memory access can be done through this register. It is out of our direct control and is automatically used. Playing with it is dangerous and needs special care. Program control instructions change the IP register.

Stack Pointer (SP)

It is a memory pointer and is used indirectly by a set of instructions. This register will be explored in the discussion of the system stack.

Base Pointer (BP)

It is also a memory pointer containing the address in a special area of memory called the stack and will be explored alongside SP in the discussion of the stack.

Flags Register

The flags register as previously discussed is not meaningful as a unit rather it is bit wise significant and accordingly each bit is named separately. The bits not named are unused. The Intel FLAGS register has its bits organized as follows:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				O	D	I	T	S	Z		A		P		C

The individual flags are explained in the following table.

C	Carry	When two 16bit numbers are added the answer can be 17 bits long or when two 8bit numbers are added the answer can be 9 bits long. This extra bit that won't fit in the target register is placed in the carry flag where it can be used and tested.
P	Parity	Parity is the number of "one" bits in a binary number. Parity is either odd or even. This information is normally used in communications to verify the integrity of data sent from the sender to the receiver.
A	Auxiliary Carry	A number in base 16 is called a hex number and can be represented by 4 bits. The collection of 4 bits is called a nibble. During addition or subtraction if a carry goes from one nibble to the next this flag is set. Carry flag is for the carry from the whole addition while auxiliary carry is the carry from the first nibble to the second.
Z	Zero Flag	The Zero flag is set if the last mathematical or logical instruction has produced a zero in its destination.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

S	Sign Flag	A signed number is represented in its two's complement form in the computer. The most significant bit (MSB) of a negative number in this representation is 1 and for a positive number it is zero. The sign bit of the last mathematical or logical operation's destination is copied into the sign flag.
T	Trap Flag	The trap flag has a special role in debugging which will be discussed later.
I	Interrupt Flag	It tells whether the processor can be interrupted from outside or not. Sometimes the programmer doesn't want a particular task to be interrupted so the Interrupt flag can be zeroed for this time. The programmer rather than the processor sets this flag since the programmer knows when interruption is okay and when it is not. Interruption can be disabled or enabled by making this bit zero or one, respectively, using special instructions.
D	Direction Flag	Specifically related to string instructions, this flag tells whether the current operation has to be done from bottom to top of the block (D=0) or from top to bottom of the block (D=1).
O	Overflow Flag	The overflow flag is set during signed arithmetic, e.g. addition or subtraction, when the sign of the destination changes unexpectedly. The actual process sets the overflow flag whenever the carry into the MSB is different from the carry out of the MSB

Segment Registers (CS, DS, SS, and ES)

The code segment register, data segment register, stack segment register, and the extra segment register are special registers related to the Intel segmented memory model and will be discussed later.

1.7. OUR FIRST PROGRAM

The first program that we will write will only add three numbers. This very simple program will clarify most of the basic concepts of assembly language. We will start with writing our algorithm in English and then moving on to convert it into assembly language.

English Language Version

"Program is an ordered set of instructions for the processor." Our first program will be instructions manipulating AX and BX in plain English.

```

move 5 to ax
move 10 to bx
add bx to ax
move 15 to bx
add bx to ax

```

Even in this simple reflection of thoughts in English, there are some key things to observe. One is the concept of destination as every instruction has a "to destination" part and there is a source before it as well. For example the second line has a constant 10 as its source and the register BX as its destination. The key point in giving the first program in English is to convey that the concepts of assembly language are simple but fine. Try to understand them considering that all above is everyday English that you know very well and every concept will eventually be applicable to assembly language.

Assembly Language Version

Intel could have made their assembly language exactly identical to our program in plain English but they have abbreviated a lot of symbols to avoid unnecessarily lengthy program when the meaning could be conveyed with less effort. For example Intel has named their move instruction “mov” instead of “move.” Similarly the Intel order of placing source and destination is opposite to what we have used in our English program, just a change of interpretation. So the Intel way of writing things is:

```
operation destination, source
operation destination
operation source
operation
```

The later three variations are for instructions that have one or both of their operands implied or they work on a single or no operand. An implied operand means that it is always in a particular register say the accumulator, and it need not be mentioned in the instruction. Now we attempt to write our program in actual assembly language of the iapx88.

Example 1.1	
001	; a program to add three numbers using registers
002	[org 0x0100]
003	mov ax, 5 ; load first number in ax
004	mov bx, 10 ; load second number in bx
005	add ax, bx ; accumulate sum in ax
006	mov bx, 15 ; load third number in bx
007	add ax, bx ; accumulate sum in ax
008	
009	mov ax, 0x4c00 ; terminate program
010	int 0x21
001	To start a comment a semicolon is used and the assembler ignores everything else on the same line. Comments must be extensively used in assembly language programs to make them readable.
002	Leave the org directive for now as it will be discussed later.
003	The constant 5 is loaded in one register AX.
004	The constant 10 is loaded in another register BX.
005	Register BX is added to register AX and the result is stored in register AX. Register AX should contain 15 by now.
006	The constant 15 is loaded in the register BX.
007	Register BX is again added to register AX now producing 15+15=30 in the AX register. So the program has computed 5+10+15=30.
008	Vertical spacing must also be used extensively in assembly language programs to separate logical blocks of code.
009-010	The ending lines are related more to the operating system than to assembly language programming. It is a way to inform DOS that our program has terminated so it can display its command prompt again. The computer may reboot or behave improperly if this termination is not present.

Assembler, Linker, and Debugger

We need an assembler to assemble this program and convert this into executable binary code. The assembler that we will use during this course is “Netwide Assembler” or NASM. It is a free and open source assembler. And the tool that will be most used will be the debugger. We will use a free debugger called “A fullscreen debugger” or AFD. These are the whole set of

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

weapons an assembly language programmer needs for any task whatsoever at hand.

To assemble we will give the following command to the processor assuming that our input file is named EX01.ASM.

```
nasm ex01.asm -o ex01.com -l ex01.lst
```

This will produce two files EX01.COM that is our executable file and EX01.LST that is a special listing file that we will explore now. The listing file produced for our example above is shown below with comments removed for neatness.

```

1
2                                [org 0x0100]
3 00000000 B80500                mov ax, 5
4 00000003 BB0A00                mov bx, 10
5 00000006 01D8                  add ax, bx
6 00000008 BB0F00                mov bx, 15
7 0000000B 01D8                  add ax, bx
8
9 0000000D B8004C                mov ax, 0x4c00
10 00000010 CD21                  int 0x21

```

The first column in the above listing is offset of the listed instruction in the output file. Next column is the opcode into which our instruction was translated. In this case this opcode is B8. Whenever we move a constant into AX register the opcode B8 will be used. After it 0500 is appended which is the immediate operand to this instruction. An immediate operand is an operand which is placed directly inside the instruction. Now as the AX register is a word sized register, and one hexadecimal digit takes 4 bits so 4 hexadecimal digits make one word or two bytes. Which of the two bytes should be placed first in the instruction, the least significant or the most significant? Similarly for 32bit numbers either the order can be most significant, less significant, lesser significant, and least significant called the big-endian order used by Motorola and some other companies or it can be least significant, more significant, more significant, and most significant called the little-endian order and is used by Intel. The big-endian have the argument that it is more natural to read and comprehend while the little-endian have the argument that this scheme places the less significant value at a lesser address and more significant value at a higher address.

Because of this the constant 5 in our instruction was converted into 0500 with the least significant byte of 05 first and the most significant byte of 00 afterwards. When read as a word it is 0005 but when written in memory it will become 0500. As the first instruction is three bytes long, the listing file shows that the offset of the next instruction in the file is 3. The opcode BB is for moving a constant into the BX register, and the operand 0A00 is the number 10 in little-endian byte order. Similarly the offsets and opcodes of the remaining instructions are shown in order. The last instruction is placed at offset 0x10 or 16 in decimal. The size of the last instruction is two bytes, so the size of the complete COM file becomes 18 bytes. This can be verified from the directory listing, using the DIR command, that the COM file produced is exactly 18 bytes long.

Now the program is ready to be run inside the debugger. The debugger shows the values of registers, flags, stack, our code, and one or two areas of the system memory as data. Debugger allows us to step our program one instruction at a time and observe its effect on the registers and program data. The details of using the AFD debugger can be seen from the AFD manual.

After loading the program in the debugger observe that the first instruction is now at 0100 instead of absolute zero. This is the effect of the org directive at the start of our program. The first instruction of a COM file must be at

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

offset 0100 (decimal 255) as a requirement. Also observe that the debugger is showing your program even though it was provided only the COM file and neither of the listing file or the program source. This is because the translation from mnemonic to opcode is reversible and the debugger mapped back from the opcode to the instruction mnemonic. This will become apparent for instructions that have two mnemonics as the debugger might not show the one that was written in the source file.

As a result of program execution either registers or memory will change. Since our program yet doesn't touch memory the only changes will be in the registers. Keenly observe the registers AX, BX, and IP change after every instruction. IP will change after every instruction to point to the next instruction while AX will accumulate the result of our addition.

1.8. SEGMENTED MEMORY MODEL

Rationale

In earlier processors like 8080 and 8085 the linear memory model was used to access memory. In linear memory model the whole memory appears like a single array of data. 8080 and 8085 could access a total memory of 64K using the 16 lines of their address bus. When designing iAPX88 the Intel designers wanted to remain compatible with 8080 and 8085 however 64K was too small to continue with, for their new processor. To get the best of both worlds they introduced the segmented memory model in 8088.

There is also a logical argument in favor of a segmented memory model in addition to the issue of compatibility discussed above. We have two logical parts of our program, the code and the data, and actually there is a third part called the program stack as well, but higher level languages make this invisible to us. These three logical parts of a program should appear as three distinct units in memory, but making this division is not possible in the linear memory model. The segmented memory model does allow this distinction.

Mechanism

The segmented memory model allows multiple functional windows into the main memory, a code window, a data window etc. The processor sees code from the code window and data from the data window. The size of one window is restricted to 64K. 8085 software fits in just one such window. It sees code, data, and stack from this one window, so downward compatibility is attained.

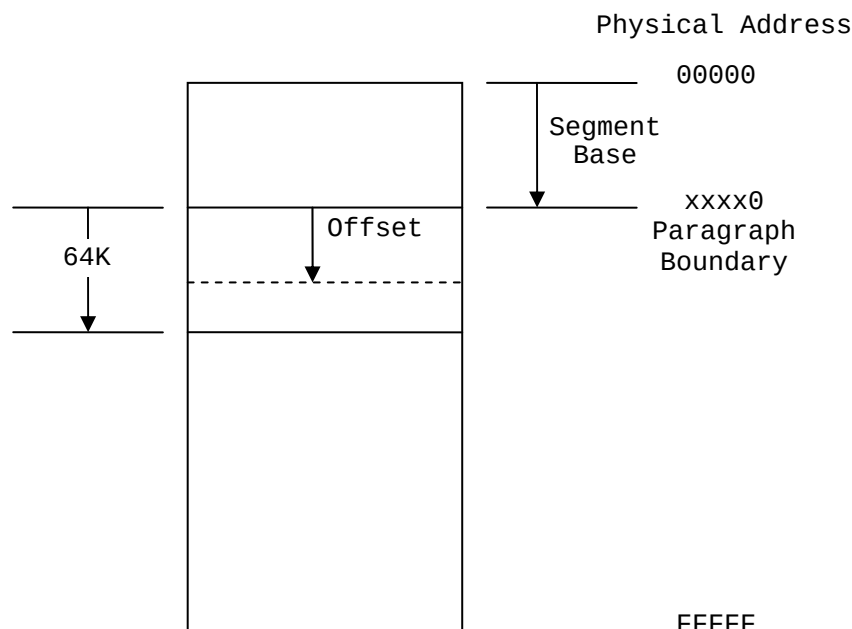
However the maximum memory iAPX88 can access is 1MB which can be accessed with 20 bits. Compare this with the 64K of 8085 that were accessed using 16 bits. The idea is that the 64K window just discussed can be moved anywhere in the whole 1MB. The four segment registers discussed in the Intel register architecture are used for this purpose. Therefore four windows can exist at one time. For example one window that is pointed to by the CS register contains the currently executing code.

To understand the concept, consider the windows of a building. We say that a particular window is 3 feet above the floor and another one is 20 feet above the floor. The reference point, the floor is the base of the segment called the datum point in a graph and all measurement is done from that datum point considering it to be zero. So CS holds the zero or the base of code. DS holds the zero of data. Or we can say CS tells how high code from the floor is, and DS tells how high data from the floor is, while SS tells how high the stack is. One extra segment ES can be used if we need to access two distant areas of memory at the same time that both cannot be seen through the same window. ES also has special role in string instructions. ES is used as an extra data segment and cannot be used as an extra code or stack segment.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

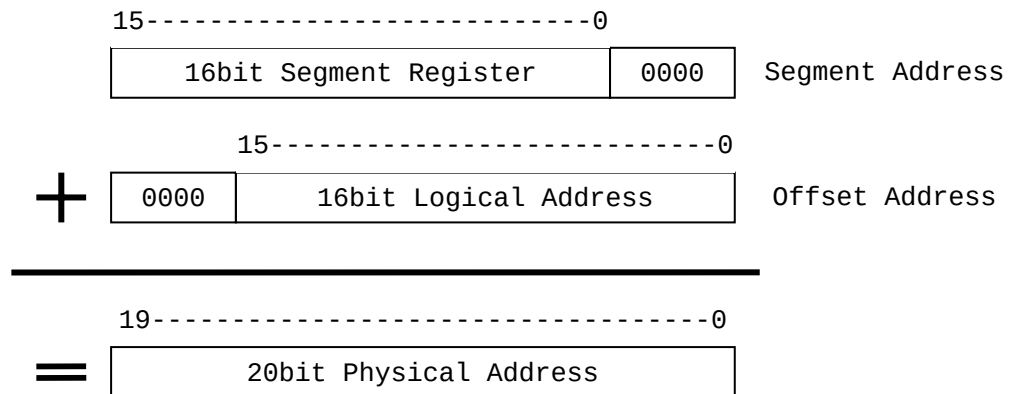
Revisiting the concept again, like the datum point of a graph, the segment registers tell the start of our window which can be opened anywhere in the megabyte of memory available. The window is of a fixed size of 64KB. Base and offset are the two key variables in a segmented address. Segment tells the base while offset is added into it. The registers IP, SP, BP, SI, DI, and BX all can contain a 16bit offset in them and access memory relative to a segment base.

The IP register cannot work alone. It needs the CS register to open a 64K window in the 1MB memory and then IP works to select code from this window as offsets. IP works only inside this window and cannot go outside of this 64K in any case. If the window is moved i.e. the CS register is changed, IP will change its behavior accordingly and start selecting from the new window. The IP register always works relatively, relative to the segment base stored in the CS register. IP is a 16bit register capable of accessing only 64K memory so how the whole megabyte can contain code anywhere. Again the same concept is there, it can access 64K at one instance of time. As the base is changed using the CS register, IP can be made to point anywhere in the whole megabyte. The process is illustrated with the following diagram.



Physical Address Calculation

Now for the whole megabyte we need 20 bits while CS and IP are both 16bit registers. We need a mechanism to make a 20bit number out of the two 16bit numbers. Consider that the segment value is stored as a 20 bit number with the lower four bits zero and the offset value is stored as another 20 bit number with the upper four bits zeroed. The two are added to produce a 20bit absolute address. A carry if generated is dropped without being stored anywhere and the phenomenon is called address wraparound. The process is explained with the help of the following diagram.



Therefore memory is determined by a segment-offset pair and not alone by any one register which will be an ambiguous reference. Every offset register is assigned a default segment register to resolve such ambiguity. For example the program we wrote when loaded into memory had a value of 0100 in IP register and some value say 1DDD in the CS register. Making both 20 bit numbers, the segment base is 1DDD0 and the offset is 00100 and adding them we get the physical memory address of 1DED0 where the opcode B80500 is placed.

Paragraph Boundaries

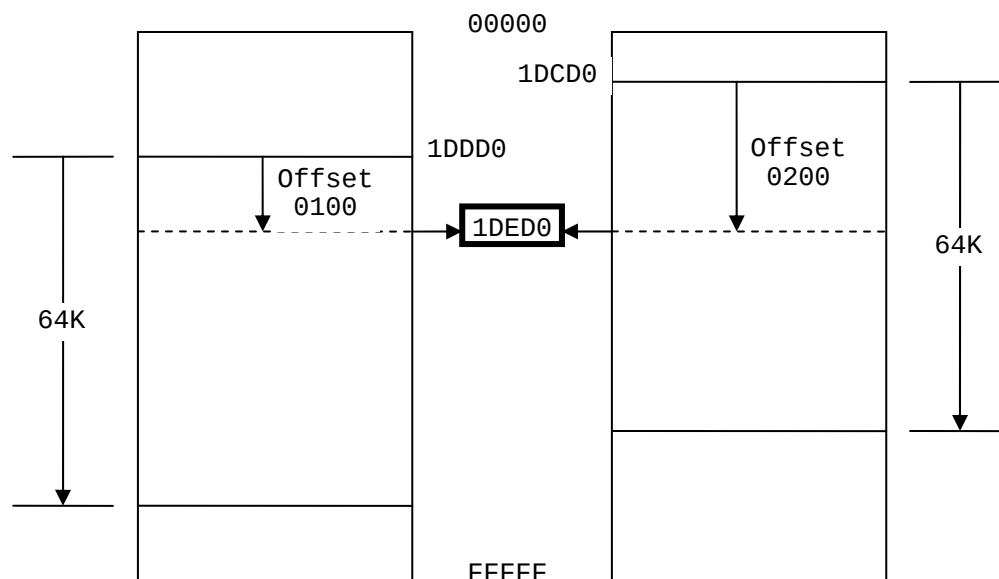
As the segment value is a 16bit number and four zero bits are appended to the right to make it a 20bit number, segments can only be defined a 16byte boundaries called paragraph boundaries. The first possible segment value is 0000 meaning a physical base of 00000 and the next possible value of 0001 means a segment base of 00010 or 16 in decimal. Therefore segments can only be defined at 16 byte boundaries.

Overlapping Segments

We can also observe that in the case of our program CS, DS, SS, and ES all had the same value in them. This is called overlapping segments so that we can see the same memory from any window. This is the structure of a COM file.

Using partially overlapping segments we can produce a number of segment, offset pairs that all access the same memory. For example 1DDD:0100 and IDED:0000 both point to the same physical memory. To test this we can open a data window at 1DED:0000 in the debugger and change the first three bytes to "90" which is the opcode for NOP (no operation). The change is immediately visible in the code window which is pointed to by CS containing 1DDD. Similarly IDCD:0200 also points to the same memory location. Consider this like a portion of wall that three different people on three different floors are seeing through their own windows. One of them painted the wall red; it will be changed for all of them though their perspective is different. It is the same phenomenon occurring here.

The segment, offset pair is called a logical address, while the 20bit address is a physical address which is the real thing. Logical addressing is a mechanism to access the physical memory. As we have seen three different logical addresses accessed the same physical address.



EXERCISES

- How the processor uses the address bus, the data bus, and the control bus to communicate with the system memory?
- Which of the following are unidirectional and which are bidirectional?
 - Address Bus
 - Data Bus
 - Control Bus
- What are registers and what are the specific features of the accumulator, index registers, program counter, and program status word?
- What is the size of the accumulator of a 64bit processor?
- What is the difference between an instruction mnemonic and its opcode?
- How are instructions classified into groups?
- A combination of 8bits is called a byte. What is the name for 4bits and for 16bits?
- What is the maximum memory 8088 can access?
- List down the 14 registers of the 8088 architecture and briefly describe their uses.
- What flags are defined in the 8088 FLAGS register? Describe the function of the zero flag, the carry flag, the sign flag, and the overflow flag.
- Give the value of the zero flag, the carry flag, the sign flag, and the overflow flag after each of the following instructions if AX is initialized with 0x1254 and BX is initialized with 0x0FFF.
 - add ax, 0xEDAB
 - add ax, bx
 - add bx, 0xF001
- What is the difference between little endian and big endian formats? Which format is used by the Intel 8088 microprocessor?
- For each of the following words identify the byte that is stored at lower memory address and the byte that is stored at higher memory address in a little endian computer.
 - 1234
 - ABFC
 - B100
 - B800

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

14. What are the contents of memory locations 200, 201, 202, and 203 if the word 1234 is stored at offset 200 and the word 5678 is stored at offset 202?
15. What is the offset at which the first executable instruction of a COM file must be placed?
16. Why was segmentation originally introduced in 8088 architecture?
17. Why a segment start cannot start from the physical address 55555.
18. Calculate the physical memory address generated by the following segment offset pairs.
 - a. 1DDD:0436
 - b. 1234:7920
 - c. 74F0:2123
 - d. 0000:6727
 - e. FFFF:4336
 - f. 1080:0100
 - g. AB01:FFFF
19. What are the first and the last physical memory addresses accessible using the following segment values?
 - a. 1000
 - b. 0FFF
 - c. 1002
 - d. 0001
 - e. E000
20. Write instructions that perform the following operations.
 - a. Copy BL into CL
 - b. Copy DX into AX
 - c. Store 0x12 into AL
 - d. Store 0x1234 into AX
 - e. Store 0xFFFF into AX
21. Write a program in assembly language that calculates the square of six by adding six to the accumulator six times.

Addressing Modes

2.1. DATA DECLARATION

The first instruction of our first assembly language program was “mov ax, 5.” Here MOV was the opcode; AX was the destination operand, while 5 was the source operand. The value of 5 in this case was stored as part of the instruction encoding. In the opcode B80500, B8 was the opcode and 0500 was the operand stored immediately afterwards. Such an operand is called an immediate operand. It is one of the many types of operands available.

Writing programs using just the immediate operand type is difficult. Every reasonable program needs some data in memory apart from constants. Constants cannot be changed, i.e. they cannot appear as the destination operand. In fact placing them as destination is meaningless and illegal according to assembly language syntax. Only registers or data placed in memory can be changed. So real data is the one stored in memory, with a very few constants. So there must be a mechanism in assembly language to store and retrieve data from memory.

To declare a part of our program as holding data instead of instructions we need a couple of very basic but special assembler directives. The first directive is “define byte” written as “db.”

```
db    somevalue
```

As a result a cell in memory will be reserved containing the desired value in it and it can be used in a variety of ways. Now we can add variables instead of constants. The other directive is “define word” or “dw” with the same syntax as “db” but reserving a whole word of 16 bits instead of a byte. There are directives to declare a double or a quad word as well but we will restrict ourselves to byte and word declarations for now. For single byte we use db and for two bytes we use dw.

To refer to this variable later in the program, we need the address occupied by this variable. The assembler is there to help us. We can associate a symbol with any address that we want to remember and use that symbol in the rest of the code. The symbol is there for our own comprehension of code. The assembler will calculate the address of that symbol using our origin directive and calculating the instruction lengths or data declarations in-between and replace all references to the symbol with the corresponding address. This is just like variables in a higher level language, where the compiler translates them into addresses; just the process is hidden from the programmer one level further. Such a symbol associated to a point in the program is called a label and is written as the label name followed by a colon.

2.2. DIRECT ADDRESSING

Now we will rewrite our first program such that the numbers 5, 10, and 15 are stored as memory variables instead of constants and we access them from there.

Example 2.1

001	; a program to add three numbers using memory variables	
002	[org 0x0100]	
003	mov ax, [num1]	; load first number in ax

004		mov	bx, [num2]	; load second number in bx
005		add	ax, bx	; accumulate sum in ax
006		mov	bx, [num3]	; load third number in bx
007		add	ax, bx	; accumulate sum in ax
008		mov	[num4], ax	; store sum in num4
009				
010		mov	ax, 0x4c00	; terminate program
011		int	0x21	
012				
013	num1:	dw	5	
014	num2:	dw	10	
015	num3:	dw	15	
016	num4:	dw	0	

002	Originate our program at 0100. The first executable instruction should be placed at this offset.
003	The source operand is changed from constant 5 to [num1]. The bracket is signaling that the operand is placed in memory at address num1. The value 5 will be loaded in ax even though we did not specified it in our program code, rather the value will be picked from memory. The instruction should be read as “read the contents of memory location num1 in the ax register.” The label num1 is a symbol for us but an address for the processor while the conversion is done by the assembler.
013	The label num1 is defined as a word and the assembler is requested to place 5 in that memory location. The colon signals that num1 is a label and not an instruction.

Using the same process to assemble as discussed before we examine the listing file generated as a result with comments removed.

1						
2			[org 0x0100]			
3	00000000	A1[1700]		mov	ax, [num1]	
4	00000003	8B1E[1900]		mov	bx, [num2]	
5	00000007	01D8		add	ax, bx	
6	00000009	8B1E[1B00]		mov	bx, [num3]	
7	0000000D	01D8		add	ax, bx	
8	0000000F	A3[1D00]		mov	[num4], ax	
9						
10	00000012	B8004C		mov	ax, 0x4c00	
11	00000015	CD21		int	0x21	
12						
13	00000017	0500	num1:	dw	5	
14	00000019	0A00	num2:	dw	10	
15	0000001B	0F00	num3:	dw	15	
16	0000001D	0000	num4:	dw	0	

The first instruction of our program has changed from B80500 to A11700. The opcode B8 is used to move constants into AX, while the opcode A1 is used when moving data into AX from memory. The immediate operand to our new instruction is 1700 or as a word 0017 (23 decimal) and from the bottom of the listing file we can observe that this is the offset of num1. The assembler has calculated the offset of num1 and used it to replace references to num1 in the whole program. Also the value 0500 can be seen at offset 0017 in the file. We can say contents of memory location 0017 are 0005 as a word. Similarly num2, num3, and num4 are placed at 0019, 001B, and 001D addresses.

When the program is loaded in the debugger, it is loaded at offset 0100, which displaces all memory accesses in our program. The instruction A11700 is changed to A11701 meaning that our variable is now placed at 0117 offset. The instruction is shown as mov ax, [0117]. Also the data window can be used to verify that offset 0117 contains the number 0005.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

Execute the program step by step and examine how the memory is read and the registers are updated, how the instruction pointer moves forward, and how the result is saved back in memory. Also observe inside the debugger code window below the code for termination, that the debugger is interpreting our data as code and showing it as some meaningless instructions. This is because the debugger sees everything as code in the code window and cannot differentiate our declared data from opcodes. It is our responsibility that we terminate execution before our data is executed as code.

Also observe that our naming of num1, num2, num3, and num4 is no longer there inside the debugger. The debugger is only showing the numbers 0117, 0119, 011B, and 011D. Our numerical machine can only work with numbers. We used symbols for our ease to label or tag certain positions in our program. The assembler converts these symbols into the appropriate numbers automatically. Also observe that the effect of “dw” is to place 5 in two bytes as 0005. Had we used “db” this would have been stored as 05 in one byte.

Given the fact that the assembler knows only numbers we can write the same program using a single label. As we know that num2 is two ahead of num1, we can use num1+2 instead of num2 and let the assembler calculate the sum during assembly process.

Example 2.2	
001	; a program to add three numbers accessed using a single label
002	[org 0x0100]
003	mov ax, [num1] ; load first number in ax
004	mov bx, [num1+2] ; load second number in bx
005	add ax, bx ; accumulate sum in ax
006	mov bx, [num1+4] ; load third number in bx
007	add ax, bx ; accumulate sum in ax
008	mov [num1+6], ax ; store sum at num1+6
009	
010	mov ax, 0x4c00 ; terminate program
011	int 0x21
012	
013	num1: dw 5
014	dw 10
015	dw 15
016	dw 0
004	The second number is read from num1+2. Similarly the third number is read from num1+4 and the result is accessed at num1+6.
013-016	The labels num2, num3, and num4 are removed and the data there will be accessed with reference to num1.

Every location is accessed with reference to num1 in this example. The expression “num1+2” comprises of constants only and can be evaluated at the time of assembly. There are no variables involved in this expression. As we open the program inside the debugger we see a verbatim copy of the previous program. There is no difference at all since the assembler catered for the differences during assembly. It calculated 0117+2=0119 while in the previous it directly knew from the value of num2 that it has to write 0119, but the end result is a ditto copy of the previous execution.

Another way to declare the above data and produce exactly same results is shown in the following example.

Example 2.3	
001	; a program to add three numbers accessed using a single label
002	[org 0x0100]
003	mov ax, [num1] ; load first number in ax
004	mov bx, [num1+2] ; load second number in bx

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

005	add ax, bx	; accumulate sum in ax
006	mov bx, [num1+4]	; load third number in bx
007	add ax, bx	; accumulate sum in ax
008	mov [num1+6], ax	; store sum at num1+6
009		
010	mov ax, 0x4c00	; terminate program
011	int 0x21	
012		
013	num1: dw 5, 10, 15, 0	
013	As we do not need to place labels on individual variables we can save space and declare all data on a single line separated by commas. This declaration will declare four words in consecutive memory locations while the address of first one is num1.	

The method used to access memory in the above examples is called direct addressing. In direct addressing the memory address is fixed and is given in the instruction. The actual data used is placed in memory and now that data can be used as the destination operand as well. Also the source and destination operands must have the same size. For example a word defined memory is read in a word sized register. A last observation is that the data 0500 in memory was corrected to 0005 when read in a register. So registers contain data in proper order as a word.

A last variation using direct addressing shows that we can directly add a memory variable and a register instead of adding a register into another that we were doing till now.

Example 2.4		
01	; a program to add three numbers directly in memory	
02	[org 0x0100]	
03	mov ax, [num1]	; load first number in ax
04	mov [num1+6], ax	; store first number in result
05	mov ax, [num1+2]	; load second number in ax
06	add [num1+6], ax	; add second number to result
07	mov ax, [num1+4]	; load third number in ax
08	add [num1+6], ax	; add third number to result
09		
10	mov ax, 0x4c00	; terminate program
11	int 0x21	
12		
13	num1: dw 5, 10, 15, 0	

We generate the following listing file as a result of the assembly process described previously. Comments are again removed.

1			
2		[org 0x0100]	
3	00000000	A1[1900]	mov ax, [num1]
4	00000003	A3[1F00]	mov [num1+6], ax
5	00000006	A1[1B00]	mov ax, [num1+2]
6	00000009	0106[1F00]	add [num1+6], ax
7	0000000D	A1[1D00]	mov ax, [num1+4]
8	00000010	0106[1F00]	add [num1+6], ax
9			
10	00000014	B8004C	mov ax, 0x4c00
11	00000017	CD21	int 0x21
12			
13	00000019	05000A000F000000	num1: dw 5, 10, 15, 0

The opcode of add is changed because the destination is now a memory location instead of a register. No other significant change is seen in the listing file. Inside the debugger we observe that few opcodes are longer now and the location num1 is now translating to 0119 instead of 0117. This is done automatically by the assembler as a result of using labels instead of

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

hard coding addresses. During execution we observe that the word data as it is read into a register is read in correct order. The significant change in this example is that the destination of addition is memory. Method to access memory is direct addressing, whether it is the MOV instruction or the ADD instruction.

The first two instructions of the last program read a number into AX and placed it at another memory location. A quick thought reveals that the following might be a possible single instruction to replace the couple.

```
mov [num1+6], [num1] ; ILLEGAL
```

However this form is illegal and not allowed on the Intel architecture. None of the general operations of mov add, sub etc. allow moving data from memory to memory. Only register to register, register to memory, memory to register, constant to memory, and constant to register operations are allowed. The other register to constant, memory to constant, and memory to memory are all disallowed. Only string instructions allow moving data from memory to memory and will be discussed in detail later. As a rule one instruction can have at most one operand in brackets, otherwise assembler will give an error.

2.3. SIZE MISMATCH ERRORS

If we change the directive in the last example from DW to DB, the program will still assemble and debug without errors, however the results will not be the same as expected. When the first operand is read 0A05 will be read in the register which was actually two operands place in consecutive byte memory locations. The second number will be read as 000F which is the zero byte of num4 appended to the 15 of num3. The third number will be junk depending on the current state of the machine. According to our data declaration the third number should be at 0114 but it is accessed at 011D calculated with word offsets. This is a logical error of the program. To keep the declarations and their access synchronized is the responsibility of the programmer and not the assembler. The assembler allows the programmer to do everything he wants to do, and that can possibly run on the processor. The assembler only keeps us from writing illegal instructions which the processor cannot execute. This is the difference between a syntax error and a logic error. So the assembler and debugger have both done what we asked them to do but the programmer asked them to do the wrong chore.

The programmer is responsible for accessing the data as word if it was declared as a word and accessing it as a byte if it was declared as a byte. The word case is shown in lot of previous examples. If however the intent is to treat it as a byte the following code shows the appropriate way.

Example 2.5	
001	; a program to add three numbers using byte variables
002	[org 0x0100]
003	mov al, [num1] ; load first number in al
004	mov bl, [num1+1] ; load second number in bl
005	add al, bl ; accumulate sum in al
006	mov bl, [num1+2] ; load third number in bl
007	add al, bl ; accumulate sum in al
008	mov [num1+3], al ; store sum at num1+3
009	
010	mov ax, 0x4c00 ; terminate program
011	int 0x21
012	
013	num1: db 5, 10, 15, 0
003	The number is read in AL register which is a byte register since the memory location read is also of byte size.
005	The second number is now placed at num1+1 instead of num1+2 because of byte offsets.

013	To declare data db is used instead of dw so that each data declared occupies one byte only.
-----	---

Inside the debugger we observe that the AL register takes appropriate values and the sum is calculated and stored in num1+3. This time there is no alignment or synchronization error. The key thing to understand here is that the processor does not match defines to accesses. It is the programmer's responsibility. In general assembly language gives a lot of power to the programmer but power comes with responsibility. Assembly language programming is not a difficult task but a responsible one.

In the above examples, the processor knew the size of the data movement operation from the size of the register involved, for example in "mov ax, [num1]" memory can be accessed as byte or as word, it has no hard and fast size, but the AX register tells that this operation has to be a word operation. Similarly in "mov al, [num1]" the AL register tells that this operation has to be a byte operation. However in "mov ax, bl" the AX register tells that the operation has to be a word operation while BL tells that this has to be a byte operation. The assembler will declare that this is an illegal instruction. A 5Kg bag cannot fit inside a 1Kg bag and according to Intel a 1Kg cannot also fit in a 5Kg bag. They must match in size. The instruction "mov [num1], [num2]" is illegal as previously discussed not because of data movement size but because memory to memory moves are not allowed at all.

The instruction "mov [num1], 5" is legal but there is no way for the processor to know the data movement size in this operation. The variable num1 can be treated as a byte or as a word and similarly 5 can be treated as a byte or as a word. Such instructions are declared ambiguous by the assembler. The assembler has no way to guess the intent of the programmer as it previously did using the size of the register involved but there is no register involved this time. And memory is a linear array and label is an address in it. There is no size associated with a label. Therefore to resolve its ambiguity we clearly tell our intent to the assembler in one of the following ways.

```
mov byte [num1], 5
mov word [num1], 5
```

2.4. REGISTER INDIRECT ADDRESSING

We have done very elementary data access till now. Assume that the numbers we had were 100 and not just three. This way of adding them will cost us 200 instructions. There must be some method to do a task repeatedly on data placed in consecutive memory cells. The key to this is the need for some register that can hold the address of data. So that we can change the address to access some other cell of memory using the same instruction. In direct addressing mode the memory cell accessed was fixed inside the instruction. There is another method in which the address can be placed in a register so that it can be changed. For the following example we will take 10 instead of 100 numbers but the algorithm is extensible to any size.

There are four registers in iAPX88 architecture that can hold address of data and they are BX, BP, SI, and DI. There are minute differences in their working which will be discussed later. For the current example, we will use the BX register and we will take just three numbers and extend the concept with more numbers in later examples.

Example 2.6		
001	; a program to add three numbers using indirect addressing	
002	[org 0x100]	
003	mov bx, num1	; point bx to first number
004	mov ax, [bx]	; load first number in ax
005	add bx, 2	; advance bx to second number

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

006	add ax, [bx]	; add second number to ax
007	add bx, 2	; advance bx to third number
008	add ax, [bx]	; add third number to ax
009	add bx, 2	; advance bx to result
010	mov [bx], ax	; store sum at num1+6
011		
012	mov ax, 0x4c00	; terminate program
013	int 0x21	
014		
015	num1: dw 5, 10, 15, 0	
003	Observe that no square brackets around num1 are used this time. The address is loaded in bx and not the contents. Value of num1 is 0005 and the address is 0117. So BX will now contain 0117.	
004	Brackets are now used around BX. In iapx88 architecture brackets can be used around BX, BP, SI, and DI only. In iapx386 more registers are allowed. The instruction will be read as “move into ax the contents of the memory location whose address is in bx.” Now since bx contains the address of num1 the contents of num1 are transferred to the ax register. Without square brackets the meaning of the instruction would have been totally different.	
005	This instruction is changing the address. Since we have words not bytes, we add two to bx so that it points to the next word in memory. BX now contains 0119 the address of the second word in memory. This was the mechanism to change addresses that we needed.	

Inside the debugger we observe that the first instruction is “mov bx, 011C.” A constant is moved into BX. This is because we did not use the square brackets around “num1.” The address of “num1” has moved to 011C because the code size has changed due to changed instructions. In the second instruction BX points to 011C and the value read in AX is 0005 which can be verified from the data window. After the addition BX points to 011E containing 000A, our next word, and so on. This way the BX register points to our words one after another and we can add them using the same instruction “mov ax, [bx]” without fixing the address of our data in the instructions. We can also subtract from BX to point to previous cells. The address to be accessed is now in total program control.

One thing that we needed in our problem to add hundred numbers was the capability to change address. The second thing we need is a way to repeat the same instruction and a way to know that the repetition is done a 100 times, a terminal condition for the repetition. For the task we are introducing two new instructions that you should read and understand as simple English language concepts. For simplicity only 10 numbers are added in this example. The algorithm is extensible to any size.

Example 2.7		
001	; a program to add ten numbers	
002	[org 0x0100]	
003	mov bx, num1	; point bx to first number
004	mov cx, 10	; load count of numbers in cx
005	mov ax, 0	; initialize sum to zero
006		
007	l1: add ax, [bx]	; add number to ax
008	add bx, 2	; advance bx to next number
009	sub cx, 1	; numbers to be added reduced
010	jnz l1	; if numbers remain add next
011		
012	mov [total], ax	; write back sum in memory
013		
014	mov ax, 0x4c00	; terminate program
015	int 0x21	
016		
017	num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50	

018	total: dw 0
006	Labels can be used on code as well. Just like data labels they remember the address at which they are used. The assembler does not differentiate between code labels and data labels. The programmer is responsible for using a data label as data and a code label as code. The label l1 in this case is the address of the following instruction.
009	SUB is the counterpart to ADD with the same rules as that of the ADD instruction.
010	JNZ stands for “jump if not zero.” NZ is the condition in this instruction. So the instruction is read as “jump to the location l1 if the zero flag is not set.” And revisiting the zero flag definition “the zero flag is set if the last mathematical or logical operation has produced a zero in its destination.” For example “mov ax, 0” will not set the zero flag as it is not a mathematical or logical instruction. However subtraction and addition will set it. Also it is set even when the destination is not a register. Now consider the subtraction immediately preceding it. As long as the CX register is non zero after this subtraction the zero flag will not be set and the jump will be taken. And jump to l1, the processor needs to be told each and everything and the destination is an important part of every jump. Just like when we ask someone to go, we mention go to this market or that house. The processor is much more logical than us and needs the destination in every instruction that asks it to go somewhere. The processor will load l1 in the IP register and resume execution from there. The processor will blindly go to the label we mention even if it contains data and not code.

The CX register is used as a counter in this example, BX contains the changing address, while AX accumulates the result. We have formed a loop in assembly language that executes until its condition remains true. Inside the debugger we can observe that the subtract instruction clears the zero flag the first nine times and sets it on the tenth time. While the jump instruction moves execution to address l1 the first nine times and to the following line the tenth time. The jump instruction breaks program flow.

The JNZ instruction is from the program control group and is a conditional jump, meaning that if the condition NZ is true (ZF=0) it will jump to the address mentioned and otherwise it will progress to the next instruction. It is a selection between two paths. If the condition is true go right and otherwise go left. Or we can say if the weather is hot, go this way, and if it is cold, go this way. Conditional jump is the most important instruction, as it gives the processor decision making capability, so it must be given a careful thought. Some processors call it branch, probably a more logical name for it, however the functionality is same. Intel chose to name it “jump.”

An important thing in the above example is that a register is used to reference memory so this form of access is called register indirect memory access. We used the BX register for it and the B in BX and BP stands for base therefore we call register indirect memory access using BX or BP, “based addressing.” Similarly when SI or DI is used we name the method “indexed addressing.” They have the same functionality, with minor differences because of which the two are called base and index. The differences will be explained later, however for the above example SI or DI could be used as well, but we would name it indexed addressing instead of based addressing.

2.5. REGISTER + OFFSET ADDRESSING

Direct addressing and indirect addressing using a single register are two basic forms of memory access. Another possibility is to use different combinations of direct and indirect references. In the above example we used BX to access different array elements which were placed consecutively in memory like an array. We can also place in BX only the array index and not the exact address and form the exact address when we are going to access the actual memory. This way the same register can be used for accessing different arrays and also the register can be used for index comparison like the following example does.

Example 2.8	
001	; a program to add ten numbers using register + offset addressing
002	[org 0x0100]
003	mov bx, 0 ; initialize array index to zero
004	mov cx, 10 ; load count of numbers in cx
005	mov ax, 0 ; initialize sum to zero
006	
007	l1: add ax, [num1+bx] ; add number to ax
008	add bx, 2 ; advance bx to next index
009	sub cx, 1 ; numbers to be added reduced
010	jnz l1 ; if numbers remain add next
011	
012	mov [total], ax ; write back sum in memory
013	
014	mov ax, 0x4c00 ; terminate program
015	int 0x21
016	
017	num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
018	total: dw 0
003	This time BX is initialized to zero instead of array base
007	The format of memory access has changed. The array base is added to BX containing array index at the time of memory access.
008	As the array is of words, BX jumps in steps of two, i.e. 0, 2, 4. Higher level languages do appropriate incrementing themselves and we always use sequential array indexes. However in assembly language we always calculate in bytes and therefore we need to take care of the size of one array element which in this case is two.

Inside the debugger we observe that the memory access instruction is shown as “mov ax, [011F+bx]” and the actual memory accessed is the one whose address is the sum of 011F and the value contained in the BX register. This form of access is of the register indirect family and is called base + offset or index + offset depending on whether BX or BP is used or SI or DI is used.

2.6. SEGMENT ASSOCIATION

All the addressing mechanisms in iAPX88 return a number called *effective address*. For example in base + offset addressing, neither the base nor the offset alone tells the desired cell in memory to be accessed. It is only after the addition is done that the processor knows which cell to be accessed. This number which came as the result of addition is called the effective address. But the effective address is just an offset and is meaningless without a segment. Only after the segment is known, we can form the physical address that is needed to access a memory cell.

We discussed the segmented memory model of iAPX88 in reasonable detail at the end of previous chapter. However during the discussion of addressing modes we have not seen the effect of segments. Segmentation is there and it's all happening relative to a segment base. We saw DS, CS, SS, and ES

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

inside the debugger. Everything is relative to its segment base, even though we have not explicitly explained its functionality. An offset alone is not complete without a segment. As previously discussed there is a default segment associated to every register which accesses memory. For example CS is associated to IP by default; rather it is tied with it. It cannot access memory in any other segment.

In case of data, there is a bit relaxation and nothing is tied. Rather there is a default association which can be overridden. In the case of register indirect memory access, if the register used is one of SI, DI, or BX the default segment is DS. If however the register used is BP the default segment used is SS. The stack segment has a very critical and fine use and there is a reason why BP is attached to SS by default. However these will be discussed in detail in the chapter on stack. IP is tied to CS while SP is tied to SS. The association of these registers cannot be changed; they are locked with no option. Others are not locked and can be changed.

To override the association for one instruction of one of the registers BX, BP, SI or DI, we use the segment override prefix. For example “mov ax, [cs:bx]” associates BX with CS for this one instruction. For the next instruction the default association will come back to act. The processor places a special byte before the instruction called a prefix, just like prefixes and suffixes in English language. No prefix is needed or placed for default association. For example for CS the byte 2E is placed and for ES the byte 26 is placed. Opcode has not changed, but the prefix byte has modified the default association to association with the desired segment register for this one instruction.

In all our examples, we never declared a segment or used it explicitly, but everything seemed to work fine. The important thing to note is that CS, DS, SS, and ES all had the same value. The value itself is not important but the fact that all had the same value is important. All four segment windows exactly overlap. Whatever segment register we use the same physical memory will be accessed. That is why everything was working without the mention of a single segment register. This is the formation of COM files in IBM PC. A single segment contains code, data, and the stack. This format is operating system dependant, in our case defined by DOS. And our operating system defines the format of COM files such that all segments have the same value. Thus the only meaningful thing that remains is the offset.

For example if BX=0100, SI=0200, and CS=1000 and the memory access under consideration is [cs:bx+si+0x0700], the effective address formed is $bx+si+0700 = 0100 + 0200 + 0700 = 0A00$. Now multiplying the segment value by 16 makes it 10000 and adding the effective address 00A00 forms the physical address 10A00.

2.7. ADDRESS WRAPAROUND

There are two types of wraparounds. One is within a single segment and the other is inside the whole physical memory. Segment wraparound occurs when during the effective address calculation a carry is generated. This carry is dropped giving the effect that when we try to access beyond the segment limit, we are actually wrapped around to the first cell in the segment. For example if BX=9100, DS=1500 and the access is [bx+0x7000] we form the effective address $9100 + 7000 = 10100$. The carry generated is dropped forming the actual effective address of 0100. Just like a circle when we reached the end we started again from the beginning. An arc at 370 degrees is the same as an arc at 10 degrees. We tried to cross the segment boundary and it pushed us back to the start. This is called segment wraparound. The physical address in the above example will be 15100.

The same can also happen at the time of physical address calculation. For example BX=0100, DS=FFF0 and the access under consideration is [bx+0x0100]. The effective address will be 0200 and the physical address will

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

be 100100. This is a 21bit answer and cannot be sent on the address bus which is 20 bits wide. The carry is dropped and just like the segment wraparound our physical memory has wrapped around at its very top. When we tried to access beyond limits the actual access is made at the very start. This second wraparound is a bit different in newer processor with more address lines but that will be explained in later chapters.

2.8. ADDRESSING MODES SUMMARY

The iAPX88 processor supports seven modes of memory access. Remember that immediate is not an addressing mode but an operand type. Operands can be immediate, register, or memory. If the operand is memory one of the seven addressing modes will be used to access it. The memory access mechanisms can also be written in the general form “base + index + offset” and we can define the possible addressing modes by saying that any one, two, or none can be skipped from the general form to form a legal memory access.

There are a few common mistakes done in forming a valid memory access. Part of a register cannot be used to access memory. Like BX is allowed to hold an address but BL or BH are not. Address is 16bit and must be contained in a 16bit register. BX-SI is not possible. The only thing that we can do is addition of a base register with an index register. Any other operation is disallowed. BS+BP and SI+DI are both disallowed as we cannot have two base or two index registers in one memory access. One has to be a base register and the other has to be an index register and that is the reason of naming them differently.

Direct

A fixed offset is given in brackets and the memory at that offset is accessed. For example “mov [1234], ax” stores the contents of the AX registers in two bytes starting at address 1234 in the current data segment. The instruction “mov [1234], al” stores the contents of the AL register in the byte at offset 1234.

Based Register Indirect

A base register is used in brackets and the actual address accessed depends on the value contained in that register. For example “mov [bx], ax” moves the two byte contents of the AX register to the address contained in the BX register in the current data segment. The instruction “mov [bp], al” moves the one byte content of the AL register to the address contained in the BP register in the current stack segment.

Indexed Register Indirect

An index register is used in brackets and the actual address accessed depends on the value contained in that register. For example “mov [si], ax” moves the contents of the AX register to the word starting at address contained in SI in the current data segment. The instruction “mov [di], ax” moves the word contained in AX to the offset stored in DI in the current data segment.

Based Register Indirect + Offset

A base register is used with a constant offset in this addressing mode. The value contained in the base register is added with the constant offset to get the effective address. For example “mov [bx+300], ax” stores the word contained in AX at the offset attained by adding 300 to BX in the current data segment. The instruction “mov [bp+300], ax” stores the word in AX to the offset attained by adding 300 to BP in the current stack segment.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

Indexed Register Indirect + Offset

An index register is used with a constant offset in this addressing mode. The value contained in the index register is added with the constant offset to get the effective address. For example “mov [si+300], ax” moves the word contained in AX to the offset attained by adding 300 to SI in the current data segment and the instruction “mov [di+300], al” moves the byte contained in AL to the offset attained by adding 300 to DI in the current data segment.

Base + Index

One base and one index register is used in this addressing mode. The value of the base register and the index register are added together to get the effective address. For example “mov [bx+si], ax” moves the word contained in the AX register to offset attained by adding BX and SI in the current data segment. The instruction “mov [bp+di], al” moves the byte contained in AL to the offset attained by adding BP and DI in the current stack segment. Observe that the default segment is based on the base register and not on the index register. This is why base registers and index registers are named separately. Other examples are “mov [bx+di], ax” and “mov [bp+si], ax.” This method can be used to access a two dimensional array such that one dimension is in a base register and the other is in an index register.

Base + Index + Offset

This is the most complex addressing method and is relatively infrequently used. A base register, an index register, and a constant offset are all used in this addressing mode. The values of the base register, the index register, and the constant offset are all added together to get the effective address. For example “mov [bx+si+300], ax” moves the word contents of the AX register to the word in memory starting at offset attained by adding BX, SI, and 300 in the current data segment. Default segment association is again based on the base register. It might be used with the array base of a two dimensional array as the constant offset, one dimension in the base register and the other in the index register. This way all calculation of location of the desired element has been delegated to the processor.

EXERCISES

1. What is a label and how does the assembler differentiate between code labels and data labels?
2. List the seven addressing modes available in the 8088 architecture.
3. Differentiate between effective address and physical address.
4. What is the effective address generated by the following instructions? Every instruction is independent of others. Initially BX=0x0100, num1=0x1001, [num1]=0x0000, and SI=0x0100
 - a. mov ax, [bx+12]
 - b. mov ax, [bx+num1]
 - c. mov ax, [num1+bx]
 - d. mov ax, [bx+si]
5. What is the effective address generated by the following combinations if they are valid. If not give reason. Initially BX=0x0100, SI=0x0010, DI=0x0001, BP=0x0200, and SP=0xFFFF
 - a. bx-si
 - b. bx-bp
 - c. bx+10
 - d. bx-10
 - e. bx+sp
 - f. bx+di
6. Identify the problems in the following instructions and correct them by replacing them with one or two instruction having the same effect.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

- a. `mov [02], [ 22]`
 - b. `mov [wordvar], 20`
 - c. `mov bx, al`
 - d. `mov ax, [si+di+100]`
7. What is the function of segment override prefix and what changes it brings to the opcode?
8. What are the two types of address wraparound? What physical address is accessed with `[BX+SI]` if `FFFF` is loaded in `BX`, `SI`, and `DS`.
9. Write instructions to do the following.
 - a. Copy contents of memory location with offset `0025` in the current data segment into `AX`.
 - b. Copy `AX` into memory location with offset `0FFF` in the current data segment.
 - c. Move contents of memory location with offset `0010` to memory location with offset `002F` in the current data segment.
10. Write a program to calculate the square of 20 by using a loop that adds 20 to the accumulator 20 times.

3

Branching

3.1. COMPARISON AND CONDITIONS

Conditional jump was introduced in the last chapter to loop for the addition of a fixed number of array elements. The jump was based on the zero flag. There are many other conditions possible in a program. For example an operand can be greater than another operand or it can be smaller. We use comparisons and boolean expressions extensively in higher level languages. They must be available in some form in assembly language, otherwise they could not possibly be made available in a higher level language. In fact they are available in a very fine and purified form.

The basic root instruction for all comparisons is `CMP` standing for compare. The operation of `CMP` is to subtract the source operand from the destination operand, updating the flags without changing either the source or the destination. `CMP` is one of the key instructions as it introduces the capability of conditional routing in the processor.

A closer thought reveals that with subtraction we can check many different conditions. For example if a larger number is subtracted from a smaller number then borrow is needed. The carry flag plays the role of borrow during the subtraction operation. And in this condition the carry flag will be set. If two equal numbers are subtracted the answer is zero and the zero flag will be set. Every significant relation between the destination and source is evident from the sign flag, carry flag, zero flag, and the overflow flag. `CMP` is meaningless without a conditional jump immediately following it.

Another important distinction at this point is the difference between signed and unsigned numbers. In unsigned numbers only the magnitude of the number is important, whereas in signed numbers both the magnitude and the sign are important. For example -2 is greater than -3 but 2 is smaller than 3. The sign has affected our comparisons.

Inside the computer signed numbers are represented in two's complement notation. In essence a number in this representation is still a number, just that now our interpretation of this number will be signed. Whether we use jump above and below or we use jump greater or less will convey our intention to the processor. The jump above and greater operations at first sight seem to be doing the same operation, and similarly below and less operations seem to be similar. However for signed numbers `JG` and `JL` will work properly and for unsigned `JA` and `JB` will work properly and not the other way around.

It is important to note that at the time of comparison, the intent of the programmer to treat the numbers as signed or unsigned is not clear. The subtraction in `CMP` is a normal subtraction. It is only after the comparison, during the conditional jump operation, that the intent is conveyed. At that time with a specific combination of flags checked the intent is satisfied.

For example a number 2 is represented in a word as 0002 while the number -2 is represented as FFFE. In a byte they would be represented as 02 and FE. Now both have the same magnitude however the different sign has caused very different representation in two's complement form. Now if the intent is to use FFFE or decimal 65534 then the same data would be placed in the word as in case of -2. In fact if -2 and 65534 are compared the processor will set the zero flag signaling that they are exactly equal. As regards an unsigned comparison the number 65534 is much greater than 2.

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

So if a JA is taken after comparing -2 in the destination with 2 in the source the jump will be taken. If however JG is used after the same comparison the jump will not be taken as it will consider the sign and with the sign -2 is smaller than 2. The key idea is that -2 and 65534 were both stored in memory in the same form. It was the interpretation that treated it as a signed or as an unsigned number.

The unsigned comparisons see the numbers as 0 being the smallest and 65535 being the largest with the order that $0 < 1 < 2 \dots < 65535$. The signed comparisons see the number -32768 which has the same memory representation as 32768 as the smallest number and 32767 as the largest with the order $-32768 < -32767 < \dots < -1 < 0 < 1 < 2 < \dots < 32767$. All the negative numbers have the same representation as an unsigned number in the range 32768 ... 65535 however the signed interpretation of the signed comparisons makes them be treated as negative numbers smaller than zero.

All meaningful situations both for signed and unsigned numbers that occur after a comparison are detailed in the following table.

DEST = SRC	ZF = 1	When the source is subtracted from the destination and both are equal the result is zero and therefore the zero flag is set. This works for both signed and unsigned numbers.
UDEST < USRC	CF = 1	When an unsigned source is subtracted from an unsigned destination and the destination is smaller, borrow is needed which sets the carry flag.
UDEST ≤ USRC	ZF = 1 OR CF = 1	If the zero flag is set, it means that the source and destination are equal and if the carry flag is set it means a borrow was needed in the subtraction and therefore the destination is smaller.
UDEST ≥ USRC	CF = 0	When an unsigned source is subtracted from an unsigned destination no borrow will be needed either when the operands are equal or when the destination is greater than the source.
UDEST > USRC	ZF = 0 AND CF = 0	The unsigned source and destination are not equal if the zero flag is not set and the destination is not smaller since no borrow was taken. Therefore the destination is greater than the source.
SDEST < SSRC	SF ≠ OF	When a signed source is subtracted from a signed destination and the answer is negative with no overflow then the destination is smaller than the source. If however there is an overflow meaning that the sign has changed unexpectedly, the meanings are reversed and a

		positive number signals that the destination is smaller.
$SDEST \leq SSRC$	$ZF = 1$ OR $SF \neq OF$	If the zero flag is set, it means that the source and destination are equal and if the sign and overflow flags differ it means that the destination is smaller as described above.
$SDEST \geq SSRC$	$SF = OF$	When a signed source is subtracted from a signed destination and the answer is positive with no overflow then the destination is greater than the source. When an overflow is there signaling that sign has changed unexpectedly, we interpret a negative answer as the signal that the destination is greater.
$SDEST > SSRC$	$ZF = 0$ AND $SF = OF$	If the zero flag is not set, it means that the signed operands are not equal and if the sign and overflow match in addition to this it means that the destination is greater than the source.

3.2. CONDITIONAL JUMPS

For every interesting or meaningful situation of flags, a conditional jump is there. For example JZ and JNZ check the zero flag. If in a comparison both operands are same, the result of subtraction will be zero and the zero flag will be set. Thus JZ and JNZ can be used to test equality. That is why there are renamed versions JE and JNE read as jump if equal or jump if not equal. They seem more logical in writing but mean exactly the same thing with the same opcode. Many jumps are renamed with two or three names for the same jump, so that the appropriate logic can be conveyed in assembly language programs. This renaming is done by Intel and is a standard for iAPX88. JC and JNC test the carry flag. For example we may need to test whether there was an overflow in the last unsigned addition or subtraction. Carry flag will also be set if two unsigned numbers are subtracted and the first is smaller than the second. Therefore the renamed versions JB, JNAE, and JNB, JAE are there standing for jump if below, jump if not above or equal, jump if not below, and jump if above or equal respectively. The operation of all jumps can be seen from the following table.

JC JB JNAE	Jump if carry Jump if below Jump if not above or equal	CF = 1	This jump is taken if the last arithmetic operation generated a carry or required a borrow. After a CMP it is taken if the unsigned destination is smaller than the unsigned source.
JNC JNB JAE	Jump if not carry Jump if not below Jump if above or equal	CF = 0	This jump is taken if the last arithmetic operation did not

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

			generated a carry or required a borrow. After a CMP it is taken if the unsigned destination is larger or equal to the unsigned source.
JE JZ	Jump if equal Jump if zero	ZF = 1	This jump is taken if the last arithmetic operation produced a zero in its destination. After a CMP it is taken if both operands were equal.
JNE JNZ	Jump if not equal Jump if not zero	ZF = 0	This jump is taken if the last arithmetic operation did not produce a zero in its destination. After a CMP it is taken if both operands were different.
JA JNBE	Jump if above Jump if not below or equal	ZF = 0 AND CF = 0	This jump is taken after a CMP if the unsigned destination is larger than the unsigned source.
JNA JBE	Jump if not above Jump if below or equal	ZF = 1 OR CF = 1	This jump is taken after a CMP if the unsigned destination is smaller than or equal to the unsigned source.
JL JNGE	Jump if less Jump if not greater or equal	SF $\neq$ OF	This jump is taken after a CMP if the signed destination is smaller than the signed source.
JNL JGE	Jump if not less Jump if greater or equal	SF = OF	This jump is taken after a CMP if the signed destination is larger than or equal to the signed source.
JG JNLE	Jump if greater Jump if not less or equal	ZF = 0 AND SF = OF	This jump is taken after a CMP if the signed destination is larger than the signed source.
JNG JLE	Jump if not greater Jump if less or equal	ZF = 1 OR SF $\neq$ OF	This jump is taken after a CMP if the signed destination is smaller than or equal to the signed source.

JO	Jump if overflow.	OF = 1	This jump is taken if the last arithmetic operation changed the sign unexpectedly.
JNO	Jump if not overflow	OF = 0	This jump is taken if the last arithmetic operation did not change the sign unexpectedly.
JS	Jump if sign	SF = 1	This jump is taken if the last arithmetic operation produced a negative number in its destination.
JNS	Jump if not sign	SF = 0	This jump is taken if the last arithmetic operation produced a positive number in its destination.
JP JPE	Jump if parity Jump if even parity	PF = 1	This jump is taken if the last arithmetic operation produced a number in its destination that has even parity.
JNP JPO	Jump if not parity Jump if odd parity	PF = 0	This jump is taken if the last arithmetic operation produced a number in its destination that has odd parity.
JCXZ	Jump if CX is zero	CX = 0	This jump is taken if the CX register is zero.

The CMP instruction sets the flags reflecting the relation of the destination to the source. This is important as when we say jump if above, then what is above what. The destination is above the source or the source is above the destination.

The JA and JB instructions are related to unsigned numbers. That is our interpretation for the destination and source operands is unsigned. The 16th bit holds data and not the sign. In the JL and JG instructions standing for jump if lower and jump if greater respectively, the interpretation is signed. The 16th bit holds the sign and not the data. The difference between them will be made clear as an elaborate example will be given to explain the difference.

One jump is special that it is not dependant on any flag. It is JCXZ, jump if the CX register is zero. This is because of the special treatment of the CX register as a counter. This jump is regardless of the zero flag. There is no counterpart or not form of this instruction.

The adding numbers example of the last chapter can be a little simplified using the compare instruction on the BX register and eliminating the need for a separate counter as below.

Example 3.1	
001	; a program to add ten numbers without a separate counter
002	[org 0x0100]
003	mov bx, 0 ; initialize array index to zero
004	mov ax, 0 ; initialize sum to zero
005	
006	l1: add ax, [num1+bx] ; add number to ax
007	add bx, 2 ; advance bx to next index
008	cmp bx, 20 ; are we beyond the last index
009	jne l1 ; if not add next number
010	
011	mov [total], ax ; write back sum in memory
012	
013	mov ax, 0x4c00 ; terminate program
014	int 0x21
015	
016	num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
017	total: dw 0
006	The format of memory access is still base + offset.
008	BX is used as the array index as well as the counter. The offset of 11th number will be 20, so as soon as BX becomes 20 just after the 10th number has been added, the addition is stopped.
009	The jump is displayed as JNZ in the debugger even though we have written JNE in our example. This is because it is a renamed jump with the same opcode as JNZ and the debugger has no way of knowing the mnemonic that we used after looking just at the opcode. Also every code and data reference that we used till now is seen in the opcode as well. However for the jump instruction we see an operand of F2 in the opcode and not 0116. This will be discussed in detail with unconditional jumps. It is actually a short relative jump and the operand is stored in the form of positive or negative offset from this instruction.

With conditional branching in hand, there are just a few small things left in assembly language that fills some gaps. Now there is just imagination and the skill to conceive programs that can make you write any program.

3.3. UNCONDITIONAL JUMP

Till now we have been placing data at the end of code. There is no such restriction and we can define data anywhere in the code. Taking the previous example, if we place data at the start of code instead of at the end and we load our program in the debugger. We can see our data placed at the start but the debugger is intending to start execution at our data. The COM file definition said that the first executable instruction is at offset 0100 but we have placed data there instead of code. So the debugger will try to interpret that data as code and showed whatever it could make up out of those opcodes.

We introduce a new instruction called JMP. It is the unconditional jump that executes regardless of the state of all flags. So we write an unconditional jump as the very first instruction of our program and jump to the next instruction that follows our data declarations. This time 0100 contains a valid first instruction of our program.

Example 3.2	
001	; a program to add ten numbers without a separate counter
002	[org 0x0100]
003	jmp start ; unconditionally jump over data
004	
005	num1: dw 10, 20, 30, 40, 50, 10, 20, 30, 40, 50
006	total: dw 0

007				
008	start:	mov	bx, 0	; initialize array index to zero
009		mov	ax, 0	; initialize sum to zero
010				
011	l1:	add	ax, [num1+bx]	; add number to ax
012		add	bx, 2	; advance bx to next index
013		cmp	bx, 20	; are we beyond the last index
014		jne	l1	; if not add next number
015				
016		mov	[total], ax	; write back sum in memory
017				
018		mov	ax, 0x4c00	; terminate program
019		int	0x21	
003	JMP jumps over the data declarations to the start label and execution resumes from there.			

3.4. RELATIVE ADDRESSING

Inside the debugger the instruction is shown as JMP 0119 and the location 0119 contains the original first instruction of the logic of our program. This jump is unconditional, it will always be taken. Now looking at the opcode we see F21600 where F2 is the opcode and 1600 is the operand to it. 1600 is 0016 in proper word order. 0119 is not given as a parameter rather 0016 is given.

This is position relative addressing in contrast to absolute addressing. It is not telling the exact address rather it is telling how much forward or backward to go from the current position of IP in the current code segment. So the instruction means to add 0016 to the IP register. At the time of execution of the first instruction at 0100 IP was pointing to the next instruction at 0103, so after adding 16 it became 0119, the desired target location. The mechanism is important to know, however all calculations in this mechanism are done by the assembler and by the processor. We just use a label with the JMP instruction and are ensured that the instruction at the target label will be the one to be executed.

3.5. TYPES OF JUMP

The three types of jump, near, short, and far, differ in the size of instruction and the range of memory they can jump to with the smallest short form of two bytes and a range of just 256 bytes to the far form of five bytes and a range covering the whole memory.

Short Jump

EB	Disp
----	------

Near Jump

E9	Disp Low	Disp High
----	----------	-----------

Far Jump

EA	IP Low	IP High	CS Low	CS High
----	--------	---------	--------	---------

Near Jump

When the relative address stored with the instruction is in 16 bits as in the last example the jump is called a near jump. Using a near jump we can jump anywhere within a segment. If we add a large number it will wrap around to

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

the lower part. A negative number actually is a large number and works this way using the wraparound behavior.

Short Jump

If the offset is stored in a single byte as in 75F2 with the opcode 75 and operand F2, the jump is called a short jump. F2 is added to IP as a signed byte. If the byte is negative the complement is negated from IP otherwise the byte is added. Unconditional jumps can be short, near, and far. The far type is yet to be discussed. Conditional jumps can only be short. A short jump can go +127 bytes ahead in code and -128 bytes backwards and no more. This is the limitation of a byte in signed representation.

Far Jump

Far jump is not position relative but is absolute. Both segment and offset must be given to a far jump. The previous two jumps were used to jump within a segment. Sometimes we may need to go from one code segment to another, and near and short jumps cannot take us there. Far jump must be used and a two byte segment and a two byte offset are given to it. It loads CS with the segment part and IP with the offset part. Execution therefore resumes from that location in physical memory. The three instructions that have a far form are JMP, CALL, and RET, are related to program control. Far capability makes intra segment control possible.

3.6. SORTING EXAMPLE

Moving ahead from our example of adding numbers we progress to a program that can sort a list of numbers using the tools that we have accumulated till now. Sorting can be ascending or descending like if the largest number comes at the top, followed by a smaller number and so on till the smallest number the sort will be called descending. The other order starting with the smallest number and ending at the largest is called ascending sort. This is a common problem and many algorithms have been developed to solve it. One simple algorithm is the bubble sort algorithm.

In this algorithm we compare consecutive numbers. If they are in required order e.g. if it is a descending sort and the first is larger than the second, then we leave them as it is and if they are not in order, we swap them. Then we do the same process for the next two numbers and so on till the last two are compared and possibly swapped.

A complete iteration is called a pass over the array. We need N passes at least in the simplest algorithm if N is the number of elements to be sorted. A finer algorithm is to check if any swap was done in this pass and stop as soon as a pass goes without a swap. The array is now sorted as every pair of elements is in order.

For example if our list of numbers is 60, 55, 45, and 58 and we want to sort them in ascending order, the first comparison will be of 60 and 55 and as the order will be reversed to 55 and 60. The next comparison will be of 60 and 45 and again the two will be swapped. The next comparison of 60 and 58 will also cause a swap. At the end of first pass the numbers will be in order of 55, 45, 58, and 60. Observe that the largest number has bubbled down to the bottom. Just like a bubble at bottom of water. In the next pass 55 and 45 will be swapped. 55 and 58 will not be swapped and 58 and 60 will also not be swapped. In the next pass there will be no swap as the elements are in order i.e. 45, 55, 58, and 60. The passes will be stopped as the last pass did not cause any swap. The application of bubble sort on these numbers is further explained with the following illustration.

State of Data	Swap Done	Swap Flag				
Pass 1		Off				
<table><tr><td>60</td><td>55</td><td>45</td><td>58</td></tr></table>	60	55	45	58	Yes	On
60	55	45	58			
<table><tr><td>55</td><td>60</td><td>45</td><td>58</td></tr></table>	55	60	45	58	Yes	On
55	60	45	58			
<table><tr><td>55</td><td>45</td><td>60</td><td>58</td></tr></table>	55	45	60	58	Yes	On
55	45	60	58			
Pass 2		Off				
<table><tr><td>55</td><td>45</td><td>58</td><td>60</td></tr></table>	55	45	58	60	Yes	On
55	45	58	60			
<table><tr><td>45</td><td>55</td><td>58</td><td>60</td></tr></table>	45	55	58	60	No	On
45	55	58	60			
<table><tr><td>45</td><td>55</td><td>58</td><td>60</td></tr></table>	45	55	58	60	No	On
45	55	58	60			
Pass 3		Off				
<table><tr><td>45</td><td>55</td><td>58</td><td>60</td></tr></table>	45	55	58	60	No	Off
45	55	58	60			
<table><tr><td>45</td><td>55</td><td>58</td><td>60</td></tr></table>	45	55	58	60	No	Off
45	55	58	60			
<table><tr><td>45</td><td>55</td><td>58</td><td>60</td></tr></table>	45	55	58	60	No	Off
45	55	58	60			
No more passes since swap flag is Off						

Example 3.3		
001 002 003 004 005 006 007 008 009 010 011 012 013 014 015 016 017 018 019 020 021 022 023 024 025 026 027 028	; sorting a list of ten numbers using bubble sort [org 0x0100] jmp start data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0 swap: db 0 start: mov bx, 0 ; initialize array index to zero mov byte [swap], 0 ; rest swap flag to no swaps loop1: mov ax, [data+bx] ; load number in ax cmp ax, [data+bx+2] ; compare with next number jbe noswap ; no swap if already in order mov dx, [data+bx+2] ; load second element in dx mov [data+bx+2], ax ; store first number in second mov [data+bx], dx ; store second number in first mov byte [swap], 1 ; flag that a swap has been done noswap: add bx, 2 ; advance bx to next index cmp bx, 18 ; are we at last index jne loop1 ; if not compare next two cmp byte [swap], 1 ; check if a swap has been done je start ; if yes make another pass mov ax, 0x4c00 ; terminate program int 0x21	

003	The jump instruction is placed to skip over data.
006	The swap flag can be stored in a register but as an example it is stored in memory and also to extend the concept at a later stage.
011-012	One element is read in AX and it is compared with the next element because memory to memory comparisons are not allowed.
013	If the JBE is changed to JB, not only the unnecessary swap on equal will be performed, there will be a major algorithmic flaw due to a logical error as in the case of equal elements the algorithm will never stop. JBE won't swap in the case of equal elements.
015-017	The swap is done using DX and AX registers in such a way that the values are crossed. The code uses the information that one of the elements is already in the AX register.
021	This time BX is compared with 18 instead of 20 even though the number of elements is same. This is because we pick an element and compare it with the next element. When we pick the 9th element we compare it with the next element and this is the last comparison, since if we pick the 10th element we will compare it with the 11th element and there is no 11th element in our case.
024-025	If a swap is done we repeat the whole process for possible more swaps.

Inside the debugger we observe that the JBE is changed to JNA due to the same reason as discussed for JNE and JNZ. The passes change the data in the same manner as we presented in our illustration above. If JBE in the code is changed to JAE the sort will change from ascending to descending. For signed numbers we can use JLE and JGE respectively for ascending and descending sort.

To clarify the difference of signed and unsigned jumps we change the data array in the last program to include some negative numbers as well. When JBE will be used on this data, i.e. with unsigned interpretation of the data and an ascending sort, the negative numbers will come at the end after the largest positive number. However JLE will bring the negative numbers at the very start of the list to bring them in proper ascending order according to a signed interpretation, even though they are large in magnitude. The data used is shown as below.

```
data:          dw 60, 55, 45, 50, -40, -35, 25, 30, 10, 0
```

This data includes some signed numbers as well. The JBE instruction will treat this data as an unsigned number and will cater only for the magnitude ignoring the sign. If the program is loaded in the debugger, the numbers will appear in their hexadecimal equivalent. The two numbers -40 and -35 are especially important as they are represented as FFD8 and FFDD. This data is not telling whether it is signed or unsigned. Our interpretation will decide whether it is a very large unsigned number or a signed number in two's complement form.

If the sorting algorithm is applied on the above data with JBE as the comparison instruction to sort in ascending order with unsigned interpretation, observe the comparisons of the two numbers FFD8 and FFDD. For example it will decide that FFDD > FFD8 since the first is larger in magnitude. At the end of sorting FFDD will be at the end of the list being declared the largest number and FFD8 will precede it to be the second largest.

If however the comparison instruction is changed to JLE and sorting is done on the same data it works similarly except on the two numbers FFDD and FFD8. This time JLE declares them to be smaller than every other number and also declares FFDD < FFD8. At the end of sorting, FFDD is

Computer Architecture & Assembly Language Programming	Course Code: CS401
CS401@vu.edu.pk	VU

declared to be the smallest number followed by FFD8 and then 0000. This is in contrast to the last example where JBE was used. This happened because JLE interpreted our data as signed numbers, and as a signed number FFDD has its sign bit on signaling that it is a negative number in two's complement form which is smaller than 0000 and every positive number. However JBE did not give any significance to the sign bit and included it in the magnitude. Therefore it declared the negative numbers to be the largest numbers.

If the required interpretation was of signed numbers the result produced by JLE is correct and if the required interpretation was of unsigned numbers the result produced by JBE is correct. This is the very difference between signed and unsigned integers in higher level languages, where the compiler takes the responsibility of making the appropriate jump depending on the type of integer used. But it is only at this level that we can understand the actual mechanism going on. In assembly language, use of proper jump is the responsibility of the programmer, to convey the intentions to use the data as signed or as unsigned.

The remaining possibilities of signed descending sort and unsigned descending sort can be done on the same lines and are left as an exercise. Other conditional jumps work in the same manner and can be studied from the reference at the end. Several will be discussed in more detail when they are used in subsequent chapters.

EXERCISES

1. Which registers are changed by the CMP instruction?
2. What are the different types of jumps available? Describe position relative addressing.
3. If AX=8FFF and BX=0FFF and "cmp ax, bx" is executed, which of the following jumps will be taken? Each part is independent of others. Also give the value of Z, S, and C flags.
 - a. jg greater
 - b. jl smaller
 - c. ja above
 - d. jb below
4. Write a program to find the maximum number and the minimum number from an array of ten numbers.
5. Write a program to search a particular element from an array using binary search. If the element is found set AX to one and otherwise to zero.
6. Write a program to calculate the factorial of a number where factorial is defined as:

$$\text{factorial}(x) = x*(x-1)*(x-2)*\dots*1$$

$$\text{factorial}(0) = 1$$